Plan de Pruebas de Software

Bosque Vivo

03/11/2026

Tabla de contenido

Plan de Pruebas de Software

1. Historial de Versiones
2. Información del Proyecto
3. Aprobaciones
4. Alcance y Estrategia de las Pruebas
5. Recursos
6. Planificación y Organización
7. Riesgos
8. Referencias
9. Glosario

Especificación de casos de prueba

Resumen
RF01 — Importar polígonos públicos
RF02 — Obtener series satelitales históricas
RF03 — Calcular el NDVI
RF04 — Comparar cobertura declarada y observada
RF05 — Calcular el índice de credibilidad
RF06 — Generar el reporte de auditoría
RF07 — Recibir consultas conversacionales
RF08 — Responder en lenguaje sencillo
RF09 — Recuperación de contraseña
Requisitos no funcionales
Interfaces externas y regresión
Correo SMTP, verificación en dos pasos y configuración
PostgreSQL, migración y despliegue

Informe de resultados de pruebas

1. Resumen
2. Resultados por suite
3. Mediciones de requisitos no funcionales
4. Pruebas de extremo a extremo

4.1 PostgreSQL y despliegue (versión 1.2)

5. Evaluación de los criterios de aceptación
6. Cómo repetir la ejecución

Registro de incidencias

Observaciones que no son defectos

Plan de Pruebas de Software

Proyecto: Bosque Vivo — Sistema de información para la auditoría de la supervivencia forestal posterior a la certificación de proyectos de carbono

Fecha: 03/11/2026

Estructura: plantilla «Formato_ieee829» del Politécnico Colombiano Jaime Isaza Cadavid, basada en IEEE Std 829. Este documento cubre el objetivo específico 4 del proyecto (probar el sistema mediante un plan de pruebas que verifique el cumplimiento de los requerimientos) y se apoya en la Especificación de Requisitos de Software (ERS, IEEE 830) del 20/10/2026.

Documentos que lo acompañan:

Documento	Contenido
`casos-de-prueba.md`	Especificación de los 62 casos de prueba (CP-01 a CP-62)
`informe-de-resultados.md`	Ejecución, métricas, mediciones de requisitos no funcionales y evaluación de los criterios de aceptación
`incidencias.md`	Registro de incidencias encontradas durante la ejecución
`evidencias/`	Salidas de las herramientas, reportes JUnit, cobertura, mediciones y capturas
`scripts/`	Scripts de prueba de extremo a extremo y de disponibilidad

1. Historial de Versiones

Fecha	Versión	Autor	Descripción
20/10/2026	0.1	Equipo Bosque Vivo	Primer plan (`docs/testing/plan-de-pruebas.md`): capas de prueba y matriz RF → pruebas, sin el formato IEEE 829
03/11/2026	1.0	Equipo Bosque Vivo	Plan completo según la plantilla IEEE 829, alineado con el ERS del 20/10/2026 (RF01–RF09 y requisitos no funcionales), casos de prueba, ejecución y resultados
03/11/2026	1.1	Equipo Bosque Vivo	Correo SMTP con verificación en dos pasos, avisos de seguridad e invitaciones; configuración automática; casos CP-53 a CP-58

Fecha	Versión	Autor	Descripción
04/11/2026	1.2	Equipo Bosque Vivo	PostgreSQL en Supabase, migración desde SQLite y despliegue en Vercel; casos CP-59 a CP-62

2. Información del Proyecto

Campo	Valor
Empresa / Organización	Politécnico Colombiano Jaime Isaza Cadavid — Ingeniería Informática, Proyecto Integrador (ODS 15)
Proyecto	Bosque Vivo
Representante de Equipo	Mateo Bermúdez Cuartas

3. Aprobaciones

Nombre y Apellido	Cargo	Departamento u Organización	Fecha	Firma
Mateo Bermúdez Cuartas	Desarrollador, representante del equipo	Politécnico Colombiano Jaime Isaza Cadavid		
Emmanuel Arango Velásquez	Desarrollador	Politécnico Colombiano Jaime Isaza Cadavid		
Miguel Ángel Calle Parra	Desarrollador	Politécnico Colombiano Jaime Isaza Cadavid		
Miguel Ángel Agudelo Tamayo	Desarrollador	Politécnico Colombiano Jaime Isaza Cadavid		
Juan David Andrade Vanegas	Desarrollador	Politécnico Colombiano Jaime Isaza Cadavid		
(Docente evaluador)	Docente del Proyecto Integrador	Politécnico Colombiano Jaime Isaza Cadavid		

4. Alcance y Estrategia de las Pruebas

El objeto de prueba es la versión de Bosque Vivo de la rama `main` del repositorio `MateoBermudez/bosque-vivo` a 03/11/2026: la API y los módulos de dominio (`backend/`), el sitio web (`web/`) y el panel Streamlit en desuso (`panel/`). Se prueban los dos sistemas de información del ERS —Verificación y Auditoría, y Asistente Conversacional— más el panel web de solo lectura con su administración de cuentas (RF09).

4.1 Elementos de Pruebas

Módulo / componente	Ubicación	Requisitos	Qué se prueba
Importación de polígonos	`backend/src/bosquevivo/registry`	RF01	Carga de GeoJSON, KML y GeoPackage; validación geométrica (auto-intersecciones); identificador y fecha de registro obligatorios; rechazo de duplicados con motivo; persistencia y metadatos (área de contabilidad de Karnik et al., 2025)
Series satelitales	`backend/src/bosquevivo/satellite`	RF02	Consulta por polígono y rango desde la fecha de certificación; filtro de nubosidad configurable (20 % por defecto); registro de incidencia cuando no hay escenas
Cálculo del NDVI	`satellite/ndvi.py`	RF03	Fórmula (NIR − Rojo)/(NIR + Rojo); descarte de valores fuera de [−1, 1]; compuesto mensual y promedio zonal
Comparación declarado–observado	`audit/compare.py`, `audit/components.py`	RF04	Alineación temporal con la fecha T0; umbrales de caída configurables; clasificación
Índice de credibilidad	`audit/index.py`, `audit/confidence.py`, `audit/config.py`	RF05	Escala 0–1 (y 0–100); categorías alta/media/baja e «Insuficiente»; confianza; reproducibilidad de los insumos
Reportes	`backend/src/bosquevivo/reports`, `web/src/app/informe`	RF06	Reporte HTML y PDF con polígono, serie NDVI, comparación, índice, fuentes y fecha; descarga desde el sitio
Asistente conversacional	`backend/src/bosquevivo/assistant`, `api/routers/whatsapp.py`, `api/routers/community.py`, `web/src/app/asistente`	RF07, RF08	Webhook firmado; consentimiento (Ley 1581); intenciones estado, alertas y ayuda; registro del polígono de interés; números autorizados; respuestas en español sencillo y de longitud acotada; chat en línea como canal alterno

Módulo / componente	Ubicación	Requisitos	Qué se prueba
Cuentas y recuperación de contraseña	`backend/src/bosquevivo/accounts`, `api/routers/auth.py`, `web/src/app/acceso`	RF09	Inicio de sesión; mensaje genérico; token de un solo uso con vencimiento; validación de la contraseña nueva; confirmación; bloqueo por intentos; invitaciones
Correo SMTP y verificación en dos pasos	`accounts/mailer.py`, `accounts/templates.py`, `accounts/service.py`	RF09, 3.3.2	Código de 6 dígitos por correo (intentos, vencimiento, reenvío); avisos de inicio de sesión, cambio de contraseña y bloqueo; detección del servidor SMTP, TLS, reintentos; buzón local sin SMTP
Configuración automática	`bosquevivo/setup.py`, `scripts/dev.*`, `scripts/start.*`	3.3.5, 3.3.6	Generación de secretos y `.env`, base y datos de demostración, lista de claves externas pendientes, modo producción
Base de datos y despliegue	`db/session.py`, `db/urls.py`, `db/migrate.py`, `jobs/dispatch.py`, `backend/index.py`, `scripts/deploy-vercel.sh`	3.3.3, 3.3.6	Suite completa en PostgreSQL, conexión por el pooler de Supabase, migración desde SQLite, API liviana en modo Vercel
Agente de IA (ampliación)	`backend/src/bosquevivo/ai`	RA01–RA09	Herramientas, validador de fidelidad, neutralidad, inyección de instrucciones, respaldo determinista (regresión con modelo guionizado)
Incidentes y mediciones	`core/incidents.py`, `measure.py`	3.3.1, 3.3.3, 3.3.5	Registro de incidentes con origen externo/interno y duración; tiempos de respuesta y de cálculo; determinismo
Sitio web	`web/`	3.1.1	Contrato de la API (zod), sesión firmada, componentes de dominio, compilación de producción

4.2 Pruebas de Regresión

Funcionalidades que no son el foco de los cambios de esta versión pero comparten componentes con ellos y se vuelven a probar en cada ejecución:

Funcionalidad (vista del usuario)	Por qué se afecta	Qué se verifica cada vez
Consultar el mapa y el detalle de un proyecto	Usa la misma API y base de datos que la importación estricta	Listado, polígono aligerado, serie y última auditoría (`test_api.py`, `test_karnik_import.py`, `backend.test.ts`)
Calcular una auditoría en lote	Comparte el servicio satelital, ahora con descarte de NDVI y registro de incidentes	`test_pipeline_batch.py`, `test_audit_service.py`, `test_golden_projects.py`
Conversación con WhatsApp (consentimiento, fotos, voz, borrado)	El flujo del asistente se reorganizó para las nuevas intenciones y la lista de números	`test_whatsapp_flow.py` completo
Chat del auditor y trazas	Comparte el agente y la sesión del área de auditores, que ahora usa cuentas	`test_chat_api.py`, `access.test.ts`, `session.test.ts`, E2E CP-37
Agente de IA	Ahora registra incidentes cuando el modelo falla	`test_agent.py`, `test_agent_tools.py`, `test_eval_harness.py`
Reportes	Se añadió la escala 0–1	`test_reports.py`
Panel Streamlit (en desuso)	Consume la misma API	`panel/tests`

Aspectos que se revisan en cada regresión: (1) toda la suite automática pasa (`pytest`, `vitest`, `ruff`, `pyright`, `eslint`, `tsc`, `next build`); (2) la cobertura del backend no baja de 80 %; (3) el lenguaje del producto sigue siendo neutro (nunca «fraude»); (4) ningún secreto queda en el repositorio. La integración continua de GitHub Actions (`.github/workflows/ci.yml`) ejecuta la regresión en cada solicitud de cambio.

4.3 Elementos a No Probar

Elemento	Razón	Riesgo asumido
Google Earth Engine real (`SATELLITE_PROVIDER=gee`)	Requiere una cuenta y un proyecto de Earth Engine del dueño; no está disponible en el entorno de pruebas	Los tiempos de cálculo con Earth Engine pueden superar los medidos con datos abiertos. Mitigación: el cálculo se probó con el proveedor de datos abiertos real (MODIS + Hansen) y la lógica de Earth Engine se aísla tras `SatelliteProvider`. Queda como CP-39 en ejecución complementaria
Envío real por WhatsApp Cloud API	Requiere credenciales de Meta del dueño (guía)	Diferencias del formato real del webhook. Mitigación: el webhook se prueba con cargas reales de la documentación de Meta, firma HMAC y el mismo flujo por el chat en línea. Queda como CP-49

Elemento	Razón	Riesgo asumido
Respuestas de Gemini real y evaluación del agente	Requiere `GEMINI_API_KEY` (guía)	Calidad de las respuestas libres no medida. Mitigación: modelo guionizado, validador de fidelidad, respuesta de respaldo y las intenciones del ERS funcionan sin IA. Queda como CP-50
Envío SMTP real de correos	Requiere una cuenta de correo del dueño (guía)	Bloqueo del proveedor de correo. Mitigación: el remitente SMTP (TLS, autenticación, reintentos) se prueba con un servidor simulado y el flujo completo —invitación, 2FA, avisos y recuperación— con el buzón local `.eml` y `AUTH_TWO_FACTOR=on`. Se verifica en el piloto con `bosquevivo mail-test` (CP-57)
Disponibilidad 95 % y ≤ 2 incidentes críticos al mes	Solo se puede medir durante el piloto, en el horario que se acuerde con el docente	Se entrega el script `scripts/disponibilidad.sh` y el comando `bosquevivo incidents`. Queda como CP-46
Comprensión con 5 personas no técnicas	Requiere usuarios reales	Mensajes poco claros. Mitigación: textos breves, revisados contra la lista de jerga. Queda como CP-29
RA03, RA06 y RA08 (modelo propio de visión, alertas en tiempo casi real, extracción de PDD)	Fuera del alcance de esta entrega (sección 2.6 del ERS)	Ninguno para el alcance comprometido
Rendimiento con miles de usuarios concurrentes	El ERS define un prototipo en modo sandbox, no un servicio en producción	No aplica

4.4 Enfoque de Pruebas (Estrategia)

Tipos de prueba.

- Funcionales: cada requisito RF01–RF09 tiene casos positivos y negativos (casos-de-prueba.md).
- De interfaces: contratos HTTP de la API (FastAPI `TestClient`), contrato backend–sitio validado con zod, webhook de WhatsApp con firma.
- No funcionales: rendimiento (comando `bosquevivo medir-rnf`), seguridad (firmas, HMAC, scrypt, tokens, secretos), fiabilidad (determinismo, incidentes), mantenibilidad (configuración), portabilidad (ejecución local y CI en Linux).
- De extremo a extremo: navegador Chromium con Playwright sobre la API y el sitio reales (`scripts/ejecutar-e2e.sh`).
- De regresión: suite completa automática en cada cambio.
- De aceptación con usuarios: piloto (pendiente).

Cuadrantes de pruebas ágiles (Marick, Crispin y Gregory).

Cuadrante	Propósito	En Bosque Vivo
Q1 — tecnología, apoyo al equipo	Pruebas unitarias y de componentes, automatizadas	108 pruebas unitarias (NDVI, índice, guardas, contraseñas, firma, privacidad) y pruebas de componentes del sitio
Q2 — negocio, apoyo al equipo	Pruebas funcionales y de ejemplos, automatizadas	179 pruebas de integración por requisito, 5 «golden» con categoría esperada, 20 verificaciones E2E
Q3 — negocio, crítica al producto	Exploratorias, usabilidad, aceptación	Revisión visual de pantallas (capturas), prueba de comprensión y piloto (pendientes con usuarios)
Q4 — tecnología, crítica al producto	Rendimiento, seguridad, fiabilidad	`medir-rnf`, determinismo, incidentes, análisis estático (`ruff`, `pyright`, `eslint`)

Pirámide de automatización. La base son las pruebas unitarias (rápidas, sin red), luego las de integración con SQLite en memoria y proveedores simulados (satélite `fake`, modelo guionizado, correo en memoria), y en la punta unas pocas pruebas E2E en navegador. Las dependencias externas (Earth Engine, Gemini, Meta, SMTP) están detrás de interfaces (`SatelliteProvider`, `LLMProvider`, `WhatsAppClient`, `Mailer`) para poder sustituirlas en las pruebas.

Proceso. (1) Se especifica el caso a partir del requisito del ERS. (2) Se automatiza en la capa más baja posible. (3) Se ejecuta la suite completa localmente y en CI. (4) Cada falla se registra como incidencia con severidad, se corrige y se vuelve a ejecutar la suite. (5) Los resultados y evidencias se guardan en `evidencias/`.

Prioridad.

Prioridad	Módulos	Motivo
1	RF05 índice, RF04 comparación, RF03 NDVI	Núcleo de la pregunta de investigación; un error invalida las conclusiones
2	RF07, RF08 asistente; RF09 cuentas, verificación en dos pasos y avisos; seguridad y Ley 1581	Datos personales y canal principal de la comunidad
3	RF01 importación, RF02 series, RF06 reporte	Entrada y salida del flujo de auditoría
4	Rendimiento, fiabilidad, portabilidad	Metas a validar durante el piloto
5	Ampliación de IA (RA)	Pendiente de aprobación del docente

4.5 Criterios de Aceptación

4.5.1 Criterios de Aceptación

Criterios para dar por completado el plan:

1. 100 % de las pruebas automáticas aprobadas (backend, sitio y panel) y análisis estático sin errores.

2. Cobertura de líneas del backend ≥ 80 %.
3. 100 % de los requisitos funcionales RF01–RF09 con al menos un caso ejecutado y aprobado.
4. 100 % de los casos de prioridad 1 y 2 ejecutados.
5. 0 incidencias abiertas de severidad crítica o alta.
6. Rendimiento: ≥ 90 % de las consultas del asistente en menos de 10 s y cálculo del índice en menos de 5 min por polígono.
7. Determinismo: mismo índice con los mismos insumos.
8. La compilación de producción del sitio termina sin errores.

Criterios del usuario para aceptar el aplicativo (ERS): el auditor puede consultar el índice, la serie y el reporte de un proyecto; la comunidad recibe respuestas cortas en español sencillo; los datos personales se tratan según la Ley 1581; y el resultado se presenta como análisis metodológico, sin atribuir fraude.

Los casos o partes que dependen de credenciales o usuarios externos (CP-29, CP-46, CP-49, CP-50 y el envío SMTP real de CP-57) se registran como «Bloqueado» y no impiden cerrar el plan; se ejecutan en el piloto.

4.6 Entregables

- Este plan de pruebas (Markdown, `.docx` y `.pdf`).
- Especificación de casos de prueba (`casos-de-prueba.md`).
- Informe de resultados con métricas y evaluación de criterios (`informe-de-resultados.md`).
- Registro de incidencias (`incidencias.md`).
- Evidencias: reportes JUnit (`backend-junit.xml`, `web-junit.xml`), salidas de `pytest` con cobertura, `vitest`, `ruff`, `pyright`, `eslint`, `next build`, mediciones de requisitos no funcionales (`rnf-fake.json`, `rnf-open.json`), resultado E2E (`e2e-resultados.json`, `e2e-salida.txt`) y capturas de pantalla.
- Scripts reutilizables: `scripts/ejecutar-e2e.sh`, `scripts/e2e.cjs`, `scripts/disponibilidad.sh` y los comandos `bosquevivo medir-rnf`, `bosquevivo mail-test` y `bosquevivo setup`.

5. Recursos

5.1 Requerimientos de Entornos – Hardware

Recurso	Especificación
Equipo de prueba	Computador personal con Windows 11 o Linux, 8 GB de RAM o más, 5 GB libres en disco (el ERS y el ADR 0002 definen ejecución local, sin servidores ni máquinas virtuales)
Red	Conexión a Internet estable para las fuentes de datos abiertos (Hansen en Google Cloud Storage, MODIS en Planetary Computer) y, en el piloto, para WhatsApp, Gemini y el túnel HTTPS
Usuario final	Cualquier teléfono con WhatsApp o un navegador web
Integración continua	Máquinas `ubuntu-latest` de GitHub Actions

5.2 Requerimientos de Entornos – Software

Software	Versión	Uso
Python	3.12	Backend y panel
uv	0.5 o superior	Entornos y dependencias de Python
Node.js	22 o superior (CI con 24)	Sitio web
Base de datos	SQLite (incluida en Python) y PostgreSQL 16	Base de pruebas en memoria, bases temporales y la misma suite contra PostgreSQL (`TEST_DATABASE_URL`, servicio de la CI); producción en Supabase
Chromium + Playwright	Versión preinstalada	Pruebas de extremo a extremo
Git y GitHub	—	Control de versiones y CI
Accesos (piloto)	—	Clave de Google AI Studio, app de Meta para WhatsApp, cuenta de Earth Engine, cuenta de correo SMTP

Datos de prueba: tres proyectos de demostración (`bosquevivo seed-demo`), polígonos sintéticos dentro de las pruebas, cargas de webhook de la documentación de Meta y los 55 polígonos reales de Karnik et al. (2025) para Colombia.

5.3 Herramientas de Pruebas Requeridas

Herramienta	Uso
pytest, pytest-cov	Pruebas unitarias, de integración, golden y del arnés de evaluación; cobertura
FastAPI TestClient	Pruebas de la API sin levantar servidor
Vitest	Pruebas del sitio (contrato zod, cliente del backend, sesión, formato)
Playwright (Chromium)	Pruebas E2E en navegador
Ruff, Pyright	Estilo y tipos del backend
ESLint, TypeScript (`tsc`), `next build`	Calidad y compilación del sitio
`bosquevivo medir-rnf`	Tiempos de respuesta, tiempos de cálculo y determinismo
`bosquevivo incidents`	Bitácora de incidentes (origen, duración, criticidad)
`bosquevivo eval`	Evaluación del agente con Gemini real (20 casos)
`scripts/disponibilidad.sh`	Disponibilidad del servicio durante el piloto
GitHub Actions	Regresión automática en cada cambio
Técnicas	Partición de equivalencia y valores límite (NDVI en [−1, 1], umbrales del índice, vencimiento del token, intentos fallidos), tablas de decisión (consentimiento y registro del asistente), pruebas basadas en riesgo y casos «golden»

5.4 Entrenamiento

Tema	Quién	Material
Ejecutar la suite y leer la cobertura	Todo el equipo	`README.md` (sección Calidad) y este plan
Proveedores simulados y modelo guionizado	Quien escriba pruebas nuevas	`backend/tests/conftest.py`, `ai/llm/fake.py`, `satellite/providers/fake.py`
Playwright para E2E	Responsable de la interfaz	`scripts/e2e.cjs`
Configurar WhatsApp, Gemini y correo para el piloto	Representante del equipo	`docs/guias`
Lenguaje neutro y Ley 1581	Todo el equipo	`docs/privacy/aviso-de-privacidad.md`, ADR 0009

6. Planificación y Organización

6.1 Matriz de Responsabilidades

Propuesta de reparto; el equipo puede ajustarla antes de firmar.

Integrante	Responsabilidades en las pruebas
Mateo Bermúdez Cuartas	Líder de pruebas: plan, integración continua, ejecución completa, RF09 (cuentas, contraseña, verificación en dos pasos y correo SMTP), E2E, registro de incidencias y configuración del piloto (WhatsApp, Gemini, correo)
Emmanuel Arango Velásquez	Sitio web: pruebas de Vitest, revisión visual de pantallas, accesibilidad, chat en línea y descarga de reportes (RF06 en el sitio)
Miguel Ángel Calle Parra	Datos geoespaciales: RF01 (importación), RF02 (series) y RF03 (NDVI); ejecución con Earth Engine real (CP-39)
Miguel Ángel Agudelo Tamayo	Analítica: RF04 (comparación), RF05 (índice y casos golden), RF06 (reportes); determinismo (CP-44)
Juan David Andrade Vanegas	Asistente: RF07 y RF08, prueba de comprensión con usuarios (CP-29), mediciones de rendimiento y disponibilidad en el piloto (CP-38, CP-46)

6.2 Cronograma

Actividad	Fecha
Plan inicial y suite automática base	20/10/2026
Implementación de RF09 y cierre de brechas del ERS	03/11/2026
Especificación de casos, ejecución completa, E2E y mediciones	03/11/2026

Actividad	Fecha
Configuración de credenciales (WhatsApp, Gemini, correo)	Semana del piloto
Piloto: CP-29, CP-46, CP-49, CP-50	Horario acordado con el docente
Cierre y firmas	Entrega final del semestre

7. Riesgos

Riesgo	Probabilidad	Impacto	Mitigación / contingencia
Cuotas de la capa gratuita de Gemini agotadas durante la evaluación	Media	Medio	Límites propios por minuto, día y usuario; respuesta de respaldo determinista; las intenciones del ERS no usan IA
Cambio de precios de WhatsApp Cloud API	Media	Medio	El piloto queda dentro de los 1.000 mensajes de servicio gratuitos al mes; el chat en línea `/asistente` reemplaza a WhatsApp sin costo
Sin acceso a Earth Engine	Media	Medio	Proveedor de datos abiertos (MODIS + Hansen) probado con datos reales
Nubosidad persistente que deja proyectos sin escenas	Alta	Medio	Se registra la incidencia y el índice sale «Insuficiente» con confianza baja, en lugar de un valor
Dependencias de desarrollo pendientes (credenciales del dueño)	Alta	Bajo	Casos marcados «Bloqueado» con guía paso a paso para ejecutarlos
Disponibilidad del equipo (semestre académico)	Media	Medio	Suite automatizada en CI; reparto por módulos
Restricciones de tiempo	Media	Medio	Priorización por riesgo (sección 4.4)
Premisa falsa: el polígono publicado no es el área de los créditos	Baja	Alto	Se usa el área de contabilidad de Karnik et al. cuando existe y se documenta
Pruebas frágiles por dependencias externas	Media	Bajo	Interfaces con dobles de prueba; las pruebas no usan red

8. Referencias

Referencia	Documento
Plan de Proyecto	`docs/planning/PLAN_IMPLEMENTACION.md`, `docs/planning/PLAN_AGENTE_IA.md`
Especificación de Requerimientos	ERS «Formato_ieee830» (Drive del proyecto, 20/10/2026); `docs/ers/ERS-Bosque-Vivo.pdf`; `docs/ers/matriz-de-trazabilidad.md`
Diseño General	`docs/architecture/arquitectura.md`
Diseño Detallado	`docs/architecture/adr/0001` a `0013`; `docs/methodology/indice-v0.md` e `indice-v1.md`
Procedimientos y estándares de Desarrollo	`README.md` (reglas del repositorio), `.pre-commit-config.yaml`, `.github/workflows/ci.yml`
Procedimientos y estándares de Pruebas	IEEE Std 829-2008 (documentación de pruebas de software); plantilla «Formato_ieee829» del Politécnico (Drive del proyecto)
Metodologías y estándares	IEEE Std 830-1998; Ley 1581 de 2012; Crispin, L. y Gregory, J. (2009). *Agile Testing*. Addison-Wesley; Cohn, M. (2009). *Succeeding with Agile* (pirámide de automatización)
Datos	Karnik et al. (2025), doi:10.1038/s41597-025-04868-2; Hansen et al. (2013), Global Forest Change

9. Glosario

Término	Definición
Caso de prueba (CP)	Conjunto de precondiciones, pasos, datos y resultado esperado para verificar un requisito
Prueba unitaria	Verifica una función o clase aislada, sin red ni base de datos externa
Prueba de integración	Verifica varios componentes juntos (API, base SQLite en memoria, servicios)
Prueba de extremo a extremo (E2E)	Recorre el sistema completo desde el navegador hasta la base de datos
Prueba de regresión	Repite pruebas ya aprobadas para comprobar que un cambio no rompió lo existente
Caso golden	Polígono de referencia con resultado esperado conocido
Doble de prueba	Componente simulado que reemplaza un servicio externo (satélite `fake`, modelo guionizado, correo en memoria)
Cobertura	Porcentaje de líneas del código ejecutadas por las pruebas
Incidencia	Diferencia entre el resultado obtenido y el esperado, registrada para su corrección

Término	Definición
Severidad	Impacto de una incidencia: crítica, alta, media o baja
Bloqueado	Caso que no puede ejecutarse por una dependencia externa
Partición de equivalencia / valores límite	Técnicas para elegir datos de prueba representativos y en los bordes de los rangos
NDVI	Índice de Vegetación de Diferencia Normalizada
T0	Fecha de inicio o certificación del proyecto, a partir de la cual se compara
Índice de credibilidad	Correspondencia entre lo declarado y lo observado, en escala de 0 a 1
HMAC	Código de autenticación de mensajes con clave; se usa para no guardar el teléfono y para firmar el webhook y la sesión
scrypt	Función de derivación de claves usada para guardar contraseñas
Token de un solo uso	Enlace de recuperación o invitación que vence y no puede reutilizarse (RF09)
Verificación en dos pasos (2FA)	Segundo factor: además de la contraseña se pide un código temporal enviado al correo
SMTP	Protocolo de envío de correo; STARTTLS y SSL cifran la conexión
JUnit XML	Formato estándar de reporte de resultados de pruebas

Especificación de casos de prueba

Acompaña al plan de pruebas. Versión 1.2 (04/11/2026): se añaden CP-59 a CP-62 (PostgreSQL en Supabase, migración y despliegue en Vercel). Versión 1.1: CP-53 a CP-58 (correo SMTP, verificación en dos pasos y configuración automática). Cada caso indica el requisito del ERS que verifica, la técnica, las precondiciones, los pasos, los datos, el resultado esperado, la prueba automática que lo ejecuta y el resultado de la ejecución del 03/11/2026.

Rutas de las pruebas: `backend/tests/...` (pytest), `web/src/...` (Vitest) y `docs/plan-de-pruebas/scripts/e2e.cjs` (Playwright, «E2E»).

Estados: **Aprobado**, **Fallido**, **Bloqueado** (depende de credenciales o usuarios externos).

Resumen

Requisito	Casos	Aprobados	Bloqueados
RF01 Importar polígonos	CP-01 a CP-06	6	0
RF02 Series satelitales	CP-07 a CP-09	3	0
RF03 NDVI	CP-10 a CP-12	3	0
RF04 Declarado vs observado	CP-13, CP-14	2	0
RF05 Índice de credibilidad	CP-15 a CP-18	4	0
RF06 Reporte	CP-19, CP-20	2	0
RF07 Consultas conversacionales	CP-21 a CP-26	6	0
RF08 Lenguaje sencillo	CP-27 a CP-29	2	1
RF09 Recuperación de contraseña	CP-30 a CP-37	8	0
3.3.1 Rendimiento	CP-38, CP-39	2	0
3.3.2 Seguridad	CP-40 a CP-43	4	0
3.3.3 Fiabilidad	CP-44, CP-45	2	0
3.3.4 Disponibilidad	CP-46	0	1
3.3.5 Mantenibilidad	CP-47	1	0
3.3.6 Portabilidad	CP-48	1	0
Interfaces externas y regresión	CP-49 a CP-52	3	1
Correo SMTP, 2FA y configuración	CP-53 a CP-58	5	1
PostgreSQL, migración y despliegue	CP-59 a CP-62	4	0
Total	**62**	**58**	**4**

RF01 — Importar polígonos públicos

CP-01 Importar un GeoJSON válido y la base de Karnik

- **Prioridad:** 3 · **Técnica:** partición de equivalencia (entrada válida)
- **Precondiciones:** base vacía.
- **Pasos:** 1) `bosquevivo import proyectos.geojson --source prueba` o `POST /api/v1/admin/import`. 2) Consultar `GET /api/v1/projects`.
- **Datos:** polígono válido con `project_id`, `project_name`, `country`, `start_year`.
- **Resultado esperado:** confirmación con el identificador interno asignado (`created_ids`); nombre, certificadora, fecha de registro, área y geometría persistidos. En la base de Karnik se conserva el área de contabilidad cuando existe.
- **Automatización:** `integration/test_registry_import.py::test_import_creates_projects`, `integration/test_api.py::test_admin_import`, `integration/test_karnik_import.py::test_conserva_nombre_registro_tipo_y_anio`, `::test_audita_el_area_de_contabilidad_cuando_existe`
- **Resultado:** Aprobado.

CP-02 Importar un archivo KML

- **Prioridad:** 3 · **Técnica:** partición de equivalencia (formato)
- **Pasos:** importar un KML con `ExtendedData` (`project_id`, `start_year`) en modo estricto.
- **Resultado esperado:** proyecto creado con identificador `VCS-900`, año 2016 y nombre del `Placemark`.
- **Automatización:** `integration/test_strict_import.py::test_kml_import`
- **Resultado:** Aprobado. Ver incidencia INC-01 sobre el formato de atributos admitido.

CP-03 Rechazar una geometría con auto-intersección

- **Prioridad:** 3 · **Técnica:** valor no válido
- **Datos:** polígono en forma de «corbatín».
- **Resultado esperado:** carga rechazada con el motivo «la geometría no es válida (Self-intersection…)». Con `--lenient` se repara y se cuenta como reparada.
- **Automatización:** `integration/test_strict_import.py::test_strict_rejects_self_intersection`, `::test_lenient_repairs_self_intersection`, `integration/test_karnik_import.py::test_repara_geometrias_invalidas`
- **Resultado:** Aprobado.

CP-04 Rechazar un identificador duplicado

- **Prioridad:** 3 · **Técnica:** tabla de decisión (existe / no existe)
- **Pasos:** importar dos veces el mismo archivo en modo estricto (CLI y API).
- **Resultado esperado:** la segunda carga crea 0 proyectos, rechaza 1 e informa «el identificador C3 ya existe en el repositorio».
- **Automatización:** `integration/test_strict_import.py::test_strict_rejects_duplicates`, `integration/test_api.py::test_admin_import`
- **Resultado:** Aprobado.

CP-05 Rechazar filas sin identificador o sin fecha de registro

- **Prioridad:** 3 · **Técnica:** partición de equivalencia (atributos mínimos)
- **Resultado esperado:** rechazo con «falta el identificador del proyecto» o «falta la fecha de registro del proyecto».
- **Automatización:** `integration/test_strict_import.py::test_strict_requires_id_and_date` (2 variantes)
- **Resultado:** Aprobado.

CP-06 Archivo inexistente o formato no soportado

- **Prioridad:** 3 · **Técnica:** valores no válidos
- **Resultado esperado:** error «no existe el archivo…» o «formato no soportado: .csv (use GeoJSON, KML, GeoPackage o Shapefile)»; archivos que no son de Karnik se rechazan.
- **Automatización:** `integration/test_registry_import.py::test_missing_file_raises`, `integration/test_strict_import.py::test_unsupported_format`, `integration/test_karnik_import.py::test_rechaza_archivos_que_no_son_de_karnik`
- **Resultado:** Aprobado.

RF02 — Obtener series satelitales históricas

CP-07 Serie desde la fecha de certificación hasta hoy

- **Prioridad:** 3
- **Precondiciones:** proyecto con T0 conocido.
- **Resultado esperado:** observaciones mensuales guardadas por sensor; Sentinel-2 no se pide antes de su ventana de operación; la ventana empieza 3 años antes de T0; recálculo reemplaza las observaciones previas.
- **Automatización:** `integration/test_satellite_service.py::test_refresh_timeseries_stores_months`, `::test_sentinel2_not_requested_before_launch_window`, `::test_refresh_replaces_previous_observations`, `integration/test_karnik_import.py::test_t0_sale_de_la_fecha_completa_de_inicio`
- **Resultado:** Aprobado. Con datos reales (proveedor `open`) ver CP-39.

CP-08 Umbral de nubosidad configurable (20 % por defecto)

- **Prioridad:** 3 · **Técnica:** valores límite (0–100)
- **Resultado esperado:** `CLOUD_MAX_PERCENT` vale 20 por defecto, acepta 35 y rechaza 150; el valor usado queda en la trazabilidad (`dataset_versions.cloud_max_percent`) y filtra `CLOUDY_PIXEL_PERCENTAGE` (Sentinel-2) y `CLOUD_COVER` (Landsat).
- **Automatización:** `unit/test_gee_support.py::test_umbral_de_nubes_por_defecto_es_20_y_se_valida`, `::test_el_umbral_de_nubes_queda_en_la_trazabilidad`
- **Resultado:** Aprobado.

CP-09 Sin escenas suficientes

- **Prioridad:** 2

- **Resultado esperado:** se registra un incidente «series_satelitales» de origen externo y severidad «aviso», y el índice sale «Insuficiente» con confianza baja en lugar de un valor.
- **Automatización:** `integration/test_incidents.py::test_missing_scenes_create_incident`, `unit/test_index.py::test_no_valid_data_is_insufficient`, `golden/test_golden_projects.py::test_insufficient_project_has_no_score_and_low_confidence`
- **Resultado:** Aprobado.

RF03 — Calcular el NDVI

CP-10 Fórmula del NDVI

- **Prioridad:** 1 · **Técnica:** valores conocidos
- **Datos:** NIR = 0,5, Rojo = 0,1 → 0,6667; NIR = Rojo → 0; NIR = Rojo = 0 → sin dato.
- **Automatización:** `unit/test_ndvi.py::test_ndvi_basic_values`
- **Resultado:** Aprobado.

CP-11 Descartar NDVI fuera de [−1, 1]

- **Prioridad:** 1 · **Técnica:** valores límite
- **Resultado esperado:** píxeles fuera de rango quedan sin dato; puntos con media fuera de rango no se guardan y se registran como escena inválida.
- **Automatización:** `unit/test_ndvi.py::test_ndvi_out_of_range_is_discarded`, `::test_invalid_points_are_detected`, `integration/test_incidents.py::test_out_of_range_points_are_discarded`
- **Resultado:** Aprobado (corrige INC-06).

CP-12 Compuesto mensual y promedio zonal

- **Prioridad:** 1
- **Resultado esperado:** las escenas de un mes se resumen con la mediana; una escena totalmente enmascarada cuenta como sin dato.
- **Automatización:** `unit/test_ndvi.py::test_monthly_composite_groups_scenes_and_uses_median`, `::test_fully_masked_scene_counts_as_no_data`
- **Resultado:** Aprobado.

RF04 — Comparar cobertura declarada y observada

CP-13 Alineación temporal con T0

- **Prioridad:** 1
- **Resultado esperado:** los años posteriores a T0 se marcan para la comparación; la pérdida anterior a T0 no penaliza; la fecha declarada revisada reemplaza la del registro; una declaración no revisada se ignora.
- **Automatización:** `unit/test_index.py::test_comparison_marks_years_after_t0`, `::test_loss_before_t0_is_ignored`, `integration/test_audit_service.py::test_declared_t0_overrides_registry_year`, `::test_unreviewed_declaration_is_ignored`
- **Resultado:** Aprobado.

CP-14 Caídas significativas y umbrales configurables

- **Prioridad:** 1 · **Técnica:** valores límite
- **Resultado esperado:** una caída abrupta del NDVI reduce la estabilidad; la pérdida posterior a T0 reduce la permanencia; los umbrales viven en `IndexConfig` y los pesos deben sumar 1.
- **Automatización:** `unit/test_index.py::test_abrupt_ndvi_drop_lowers_stability_and_trend_is_flat`, `::test_loss_after_t0_lowers_permanence`, `::test_weights_must_sum_to_one`, `integration/test_audit_service.py::test_forest_loss_and_ndvi_drop_reduce_score`
- **Resultado:** Aprobado.

RF05 — Calcular el índice de credibilidad

CP-15 Escala de 0 a 1

- **Prioridad:** 1
- **Resultado esperado:** el índice se muestra como «0,82» además de 82/100 en el sitio y en el reporte.
- **Automatización:** `web/src/lib/format.test.ts` («índice en escala de 0 a 1»), `integration/test_reports.py`
- **Resultado:** Aprobado.

CP-16 Categorías alta, media, baja e insuficiente

- **Prioridad:** 1 · **Técnica:** valores límite (75 y 50)
- **Resultado esperado:** bosque intacto → alta; con pérdida → menor; sin datos → «Insuficiente»; v1 distingue proyectos que v0 deja en 100.
- **Automatización:** `unit/test_index.py::test_intact_forest_scores_high`, `unit/test_index_v1.py` (13 pruebas), `golden/test_golden_projects.py::test_expected_category`, `::test_intact_scores_higher_than_damaged`
- **Resultado:** Aprobado.

CP-17 Proyectos golden

- **Prioridad:** 1
- **Resultado esperado:** tres polígonos de referencia obtienen la categoría esperada.
- **Automatización:** `golden/test_golden_projects.py`
- **Resultado:** Aprobado.

CP-18 Reproducibilidad de los insumos

- **Prioridad:** 1
- **Resultado esperado:** cada corrida guarda parámetros, versión del índice, versión del código (`git_sha`) y conjuntos de datos; el rango de sensibilidad acota el puntaje.
- **Automatización:** `integration/test_audit_service.py::test_audit_run_persists_result_and_traceability`, `unit/test_index.py::test_sensitivity_range_brackets_score`
- **Resultado:** Aprobado.

RF06 — Generar el reporte de auditoría

CP-19 Contenido del reporte HTML y PDF

- **Prioridad:** 3
- **Resultado esperado:** incluye polígono, serie NDVI, comparación, índice (0–100 y 0–1), confianza, fuentes satelitales, fecha de generación y limitaciones; escapa el nombre del proyecto; no usa lenguaje acusatorio; el PDF es válido.
- **Automatización:** `integration/test_reports.py` (5 pruebas)
- **Resultado:** Aprobado.

CP-20 Descarga desde el sitio

- **Prioridad:** 3
- **Resultado esperado:** `/informe/[runId]` entrega el archivo con su tipo de contenido o 404 si no existe.
- **Automatización:** `web/src/lib/api/backend.test.ts` («devuelve el reporte con su tipo de contenido, o null si no existe»)
- **Resultado:** Aprobado.

RF07 — Recibir consultas conversacionales

CP-21 Primer contacto

- **Prioridad:** 2
- **Pasos:** escribir «Hola» por WhatsApp o en el chat en línea.
- **Resultado esperado:** saludo que aclara que es una IA y aviso de privacidad que pide ACEPTO; no se llama al modelo.
- **Automatización:** `integration/test_whatsapp_flow.py::test_first_contact_sends_welcome_and_notice_without_calling_model`, `integration/test_community_chat.py::test_web_chat_follows_consent_and_intents`, E2E CP-21
- **Resultado:** Aprobado.

CP-22 Sin autorización no se responde

- **Prioridad:** 2 · **Técnica:** tabla de decisión (consentimiento)
- **Resultado esperado:** antes de ACEPTO cualquier consulta recibe «Escriba ACEPTO para continuar»; el consentimiento queda registrado con la versión del aviso.
- **Automatización:** `integration/test_whatsapp_flow.py::test_questions_before_consent_are_not_answered`, `::test_consent_is_logged_with_notice_version`, E2E CP-22
- **Resultado:** Aprobado.

CP-23 Intención «ayuda»

- **Prioridad:** 2
- **Resultado esperado:** AYUDA (o «menú») lista ESTADO, ALERTAS, PROYECTO y BORRAR MIS DATOS sin llamar al modelo.

- **Automatización:** `integration/test_whatsapp_intents.py::test_help_without_model`, `::test_detect_intent`, E2E CP-23
- **Resultado:** Aprobado.

CP-24 Registro del polígono de interés

- **Prioridad:** 2 · **Técnica:** partición de equivalencia (número válido / inexistente / ubicación)
- **Resultado esperado:** «PROYECTO 1» registra el proyecto; un número inexistente responde «No encontré ese número»; una ubicación dentro de un polígono lo asocia.
- **Automatización:** `integration/test_whatsapp_intents.py::test_register_then_status_and_alerts`, `::test_location_registers_project`, E2E CP-24
- **Resultado:** Aprobado.

CP-25 Intención «alertas»

- **Prioridad:** 2 · **Técnica:** valores límite (0,5 % del bosque por año)
- **Resultado esperado:** sin proyecto registrado pide registrarlo; con pérdida ≥ 0,5 % anual en los últimos 3 años informa año, hectáreas y porcentaje; sin pérdida informa «Sin alertas».
- **Automatización:** `integration/test_whatsapp_intents.py::test_alerts_require_project`, `::test_alert_thresholds`, `::test_alerts_without_loss_records`, E2E CP-25
- **Resultado:** Aprobado (corrige INC-04).

CP-26 Intención no reconocida

- **Prioridad:** 2
- **Resultado esperado:** si la consulta no se reconoce y el agente no tiene datos, se responde con la ayuda en lugar de fallar en silencio, también sin clave de IA.
- **Automatización:** `integration/test_whatsapp_intents.py::test_unknown_question_without_model_gets_help`, `integration/test_community_chat.py::test_works_without_gemini_key`, E2E CP-26
- **Resultado:** Aprobado (corrige INC-07).

RF08 — Responder en lenguaje sencillo

CP-27 Estado del bosque en términos de cobertura

- **Prioridad:** 2
- **Resultado esperado:** «En Reserva El Roble se conserva el 100 % del bosque que había desde 2018. En general, lo que se ve desde el satélite coincide bien con lo que el proyecto declara.»
- **Automatización:** `integration/test_whatsapp_intents.py::test_register_then_status_and_alerts`, E2E CP-27
- **Resultado:** Aprobado.

CP-28 Sin jerga, sin acusaciones y mensajes cortos

- **Prioridad:** 2
- **Resultado esperado:** ningún mensaje supera 300 caracteres; no aparecen «fraude», «NDVI», «REDD» ni «índice» en las respuestas a la comunidad; el agente bloquea lenguaje acusatorio.

- **Automatización:** `integration/test_whatsapp_intents.py::test_register_then_status_and_alerts`, `integration/test_whatsapp_flow.py::test_answer_after_consent_respects_length`, `unit/test_guardrails.py::test_accusatory_terms_are_detected`, `integration/test_agent.py::test_accusatory_language_is_blocked`, `::test_community_answer_longer_than_limit_falls_back`, E2E CP-28
- **Resultado:** Aprobado.

CP-29 Comprensión con usuarios no técnicos

- **Prioridad:** 2 · **Técnica:** prueba de usabilidad
- **Pasos:** 5 personas sin formación técnica usan el asistente con un guion de 5 tareas (saludo, aceptar, registrar proyecto, estado, alertas) y explican con sus palabras lo que entendieron.
- **Resultado esperado:** al menos 4 de 5 personas interpretan correctamente el estado y las alertas.
- **Resultado:** Bloqueado (requiere el piloto con usuarios).

RF09 — Recuperación de contraseña

CP-30 Acceso restringido

- **Prioridad:** 2
- **Resultado esperado:** `/chat` y `/trazas` redirigen a `/acceso`; la API del chat responde 401 sin sesión.
- **Automatización:** `web/src/lib/auth/access.test.ts`, E2E CP-30
- **Resultado:** Aprobado.

CP-31 Credenciales inválidas y bloqueo

- **Prioridad:** 2 · **Técnica:** valores límite (5 intentos, 10 minutos)
- **Resultado esperado:** «El correo o la contraseña no son correctos.»; tras 5 fallos la cuenta se bloquea 10 minutos y luego se libera.
- **Automatización:** `integration/test_password_recovery.py::test_authenticate_and_lock_after_failures`, `::test_login_endpoint`, `web/src/lib/auth/throttle.test.ts`, E2E CP-31
- **Resultado:** Aprobado.

CP-32 Mensaje genérico

- **Prioridad:** 2 · **Técnica:** partición (correo registrado / no registrado / inactivo)
- **Resultado esperado:** el mismo texto «Si la dirección está registrada, recibirás un correo…» en los tres casos; solo se envía correo a cuentas activas.
- **Automatización:** `integration/test_password_recovery.py::test_unknown_email_gets_same_message_and_no_mail`, `::test_reset_endpoints_full_flow`, `::test_disabled_user_cannot_login_or_reset`, E2E CP-32
- **Resultado:** Aprobado.

CP-33 Correo con enlace seguro de un solo uso

- **Prioridad:** 2
- **Resultado esperado:** el correo contiene `WEB_BASE_URL/acceso/restablecer?token=…` y el vencimiento (30 minutos); el token se guarda solo como SHA-256; una nueva solicitud invalida la anterior; máximo

3 solicitudes por hora; SMTP con STARTTLS y autenticación.
- **Automatización:** `integration/test_password_recovery.py::test_reset_flow_sends_single_use_link`, `::test_token_is_stored_hashed`, `::test_new_request_invalidates_previous_token`, `::test_reset_requests_are_throttled`, `::test_smtp_mailer_uses_tls_and_login`, `::test_outbox_mailer_writes_eml`, E2E CP-33
- **Resultado:** Aprobado.

CP-34 Token inválido, vencido o usado

- **Prioridad:** 2 · **Técnica:** valores límite (minuto 31)
- **Resultado esperado:** «El enlace no es válido», «El enlace venció» o «El enlace ya se usó», con la invitación a solicitar uno nuevo (HTTP 410).
- **Automatización:** `integration/test_password_recovery.py::test_token_expires`, `::test_invalid_token_and_weak_password`, `::test_reset_endpoints_full_flow`, `web/src/lib/api/accounts.test.ts`, E2E CP-34
- **Resultado:** Aprobado.

CP-35 Validación de la contraseña nueva

- **Prioridad:** 2 · **Técnica:** partición de equivalencia
- **Datos:** contraseñas distintas, menos de 10 caracteres, sin número, sin letra, igual al correo.
- **Resultado esperado:** rechazo con el motivo; el token sigue válido.
- **Automatización:** `integration/test_password_recovery.py::test_password_policy`, `::test_invalid_token_and_weak_password`, `web/src/lib/auth/throttle.test.ts`, E2E CP-35
- **Resultado:** Aprobado.

CP-36 Actualización y confirmación

- **Prioridad:** 2
- **Resultado esperado:** «Tu contraseña se actualizó. Ya puedes iniciar sesión.»; la contraseña anterior deja de funcionar.
- **Automatización:** `integration/test_password_recovery.py::test_reset_flow_sends_single_use_link`, E2E CP-36
- **Resultado:** Aprobado.

CP-37 Inicio de sesión con la contraseña nueva

- **Prioridad:** 2
- **Resultado esperado:** se entra a `/trazas`; la sesión es una cookie firmada que guarda el correo y no puede alterarse.
- **Automatización:** `web/src/lib/auth/session.test.ts`, E2E CP-37
- **Resultado:** Aprobado.

Requisitos no funcionales

CP-38 Tiempo de respuesta del asistente (3.3.1)

- **Meta:** ≥ 90 % de las consultas en menos de 10 s.

- **Pasos:** `uv run bosquevivo medir-rnf --rounds 100`.
- **Resultado obtenido:** 100 % bajo 10 s (p90 = 0,005 s) en las intenciones del ERS; con Gemini real se repite en el piloto.
- **Automatización:** `integration/test_measure.py`; evidencia `evidencias/rnf-fake.json`, `rnf-open.json`.
- **Resultado:** Aprobado.

CP-39 Tiempo de cálculo del índice por polígono (3.3.1)

- **Meta:** < 5 min por polígono.
- **Resultado obtenido:** con datos abiertos reales (MODIS 250 m + Hansen) 43,7 s, 44,7 s y 62,9 s en los tres polígonos de demostración (máximo 62,9 s).
- **Evidencia:** `evidencias/rnf-open.json`
- **Resultado:** Aprobado. Repetir con `SATELLITE_PROVIDER=gee` cuando haya cuenta de Earth Engine.

CP-40 Firma del webhook y token de administración (3.3.2)

- **Resultado esperado:** un POST sin firma `X-Hub-Signature-256` válida responde 401; la verificación del webhook solo devuelve el desafío con el token correcto; las rutas de administración, autenticación, chat y chat comunitario exigen `X-Admin-Token`.
- **Automatización:** `integration/test_whatsapp_flow.py::test_webhook_rejects_bad_signature`, `::test_webhook_verification_handshake`, `unit/test_whatsapp_support.py::test_tampered_body_or_missing_header_is_rejected`, `integration/test_password_recovery.py::test_auth_endpoints_require_admin_token`, `integration/test_community_chat.py::test_requires_admin_token`, `integration/test_api.py::test_request_audit_requires_admin_token`
- **Resultado:** Aprobado.

CP-41 Teléfono como HMAC y números autorizados (3.3.2, RF07)

- **Resultado esperado:** el número nunca se guarda en claro; con `WHATSAPP_ALLOWED_PHONES` solo se atiende a los números listados y no se crea registro de los demás.
- **Automatización:** `integration/test_whatsapp_flow.py::test_phone_is_stored_only_as_hmac`, `unit/test_privacy.py`, `integration/test_whatsapp_intents.py::test_allowlist`, `::test_unauthorized_phone_is_not_stored`
- **Resultado:** Aprobado.

CP-42 Supresión de datos (Ley 1581)

- **Resultado esperado:** BORRAR MIS DATOS elimina el usuario, los consentimientos, las consultas y la evidencia asociada.
- **Automatización:** `integration/test_whatsapp_flow.py::test_delete_command_removes_user_consents_and_traces`, `::test_delete_command_also_removes_evidence_of_the_user_queries`, `integration/test_community_chat.py::test_web_chat_follows_consent_and_intents`, E2E CP-42
- **Resultado:** Aprobado.

CP-43 Credenciales protegidas

- **Resultado esperado:** contraseñas con scrypt y sal (dos hashes de la misma clave difieren); tokens de recuperación como SHA-256; secretos solo en `.env` (ignorado por git); el agente bloquea la fuga de secretos.
- **Automatización:** `integration/test_password_recovery.py::test_password_hash_roundtrip_and_salt`, `::test_token_is_stored_hashed`, `integration/test_agent.py::test_secret_leak_in_answer_is_blocked`; revisión de `.gitignore`.
- **Resultado:** Aprobado.

CP-44 Determinismo (3.3.3)

- **Resultado esperado:** con los mismos insumos y parámetros el índice es idéntico.
- **Resultado obtenido:** 3 de 3 proyectos con el mismo índice en dos cálculos (97,2/97,2; 96,0/96,0; 56,1/56,1 con datos abiertos).
- **Automatización:** `integration/test_measure.py::test_measure_audits_and_determinism`; evidencia `rnf-open.json`.
- **Resultado:** Aprobado.

CP-45 Registro de incidentes (3.3.3, 3.3.5)

- **Resultado esperado:** cada incidente guarda componente, origen (externo/interno), severidad, inicio, fin y duración; más de 60 min se marca crítico; los fallos de tareas, del modelo y del correo se registran; `bosquevivo incidents` los resume.
- **Automatización:** `integration/test_incidents.py` (5 pruebas), `integration/test_password_recovery.py::test_mail_failure_is_recorded_as_external_incident`
- **Resultado:** Aprobado.

CP-46 Disponibilidad del 95 % en el horario del piloto (3.3.4)

- **Pasos:** durante el horario acordado, `scripts/disponibilidad.sh http://127.0.0.1:8000/health <minutos>`; al final `bosquevivo incidents`.
- **Resultado esperado:** ≥ 95 % de minutos con `/health` en 200 y ≤ 2 incidentes críticos al mes.
- **Resultado:** Bloqueado (se mide en el piloto).

CP-47 Parámetros como configuración (3.3.5)

- **Resultado esperado:** umbral de nubosidad, umbrales de caída del NDVI, pesos y categorías del índice, vencimiento del token y lista de números se cambian por configuración, sin tocar el código.
- **Automatización:** `unit/test_gee_support.py::test_umbral_de_nubes_por_defecto_es_20_y_se_valida`, `unit/test_index_v1.py::test_version_desconocida_se_rechaza`, `integration/test_password_recovery.py::test_token_expires`
- **Resultado:** Aprobado.

CP-48 Portabilidad (3.3.6)

- **Resultado esperado:** el sistema se instala y ejecuta con herramientas abiertas en un entorno local (Linux en CI y en esta ejecución; Windows con `scripts/dev.ps1`); la migración ligera actualiza bases

existentes.
- **Automatización:** CI de GitHub Actions; `integration/test_whatsapp_intents.py::test_upgrade_adds_missing_column`; `integration/test_lifespan.py`
- **Resultado:** Aprobado.

Interfaces externas y regresión

CP-49 WhatsApp Cloud API real

- **Pasos:** seguir `docs/guias/whatsapp.md` y repetir CP-21 a CP-28 desde un teléfono de prueba.
- **Resultado:** Bloqueado (requiere credenciales de Meta del dueño).

CP-50 Evaluación del agente con Gemini real

- **Pasos:** `uv run bosquevivo eval --output data/eval/report.json` (20 casos: herramientas, fidelidad, neutralidad, incertidumbre, fugas, longitud y latencia).
- **Verificación en producción (05/11/2026):** con la clave configurada, el chat del auditor respondió con datos reales usando `buscar_proyecto`, `obtener_auditoria`, `obtener_perdida_anual` y `buscar_conocimiento`, sin respaldo, en 3–5 s por pregunta (tras corregir INC-17 e INC-18); el chat comunitario respondió una pregunta libre en 2 s.
- **Resultado:** Aprobado en producción. La evaluación formal de 20 casos (`bosquevivo eval`) queda para registrar su informe.

CP-51 Regresión del agente de IA (RA01–RA09)

- **Resultado esperado:** uso de herramientas, corrección de cifras inventadas, respaldo ante cuota agotada, separación de herramientas por rol, inyección de instrucciones, límite de pasos y límite diario.
- **Automatización:** `integration/test_agent.py` (16 pruebas), `integration/test_agent_tools.py`, `integration/test_knowledge.py`, `integration/test_chat_api.py`, `unit/test_guardrails.py`
- **Resultado:** Aprobado.

CP-52 Regresión del sitio y del panel

- **Resultado esperado:** `eslint`, `tsc`, 94 pruebas de Vitest, `next build` y 5 pruebas del panel sin errores.
- **Evidencia:** `evidencias/web-vitest.txt`, `web-junit.xml`, `web-lint.txt`, `web-build.txt`
- **Resultado:** Aprobado.

Correo SMTP, verificación en dos pasos y configuración

CP-53 Verificación en dos pasos por correo

- **Requisito:** RF09 y 3.3.2 (seguridad del panel web) · **Prioridad:** 2
- **Precondiciones:** cuenta activa; SMTP configurado (o `AUTH_TWO_FACTOR=on`).
- **Pasos:** iniciar sesión con correo y contraseña correctos; leer el correo; escribir el código.
- **Resultado esperado:** tras la contraseña se pide un código de 6 dígitos enviado al correo (mostrado enmascarado, «au*****@elpoli.edu.co»); con el código correcto se entra. Una contraseña incorrecta

no envía código. Con `AUTH_TWO_FACTOR=auto` se activa solo cuando hay SMTP.
- **Automatización:** `integration/test_two_factor.py::test_login_sends_code_and_verifies`, `::test_wrong_password_does_not_send_code`, `::test_two_factor_can_be_disabled`, `::test_auto_two_factor_follows_smtp`, `::test_two_factor_endpoints`, `web/src/lib/api/accounts.test.ts`, E2E CP-53
- **Resultado:** Aprobado.

CP-54 Código incorrecto, vencido, reutilizado y reenvío

- **Prioridad:** 2 · **Técnica:** valores límite (5 intentos, 10 minutos, 3 envíos)
- **Resultado esperado:** código incorrecto → «El código no es correcto.»; al quinto fallo, al vencer o al reutilizarse, la verificación se anula y hay que iniciar sesión de nuevo; «Reenviar código» envía uno nuevo e invalida el anterior, hasta 3 envíos.
- **Automatización:** `integration/test_two_factor.py::test_wrong_code_counts_attempts_until_exhausted`, `::test_code_expires_and_is_single_use`, `::test_resend_replaces_code_with_limit`, E2E CP-54
- **Resultado:** Aprobado.

CP-55 Avisos de seguridad por correo

- **Prioridad:** 2
- **Resultado esperado:** correo de nuevo inicio de sesión (fecha en hora de Colombia, navegador, sistema e IP), de contraseña cambiada y de cuenta bloqueada; todos con versión de texto y HTML. Una falla de envío de un aviso no impide la operación y queda como incidente externo.
- **Automatización:** `integration/test_two_factor.py::test_lock_sends_security_email`, `::test_password_change_notifies`, `::test_message_has_text_and_html`, `integration/test_password_recovery.py::test_mail_failure_is_recorded_as_external_incident`, E2E CP-55
- **Resultado:** Aprobado.

CP-56 Invitación de cuentas

- **Prioridad:** 2
- **Pasos:** `bosquevivo user-invite correo --name Nombre`; abrir el enlace del correo; elegir contraseña.
- **Resultado esperado:** la cuenta se crea sin contraseña utilizable; el correo trae un enlace de un solo uso válido 72 horas; la página da la bienvenida y, al guardar, se puede iniciar sesión. No se envía el aviso de «contraseña cambió».
- **Automatización:** `integration/test_two_factor.py::test_invitation_sets_first_password`, `::test_verify_reports_purpose`, E2E CP-56
- **Resultado:** Aprobado (corrige INC-12).

CP-57 Configuración SMTP

- **Prioridad:** 3
- **Resultado esperado:** con solo `SMTP_USER` y `SMTP_PASSWORD` se detecta el servidor (Gmail, Google Workspace, Outlook, Yahoo); `MAIL_BACKEND=auto` usa SMTP si hay credenciales y el buzón local si no; STARTTLS en 587 y SSL en 465; un error transitorio se reintenta; un error de autenticación explica que se use una contraseña de aplicación; `bosquevivo mail-test` verifica la configuración.

- **Automatización:** `integration/test_two_factor.py::test_smtp_presets_and_auto_backend`, `::test_smtp_retries_and_explains_auth_errors`, `integration/test_password_recovery.py::test_smtp_mailer_uses_tls_and_login`, `::test_build_mailer_validates_configuration`
- **Resultado:** Aprobado. El envío por un servidor SMTP real se ejecuta en el piloto con `bosquevivo mail-test` (Bloqueado hasta tener las credenciales).

CP-58 Configuración automática

- **Requisito:** 3.3.5 y 3.3.6 · **Prioridad:** 3
- **Pasos:** en un clon nuevo, `scripts/dev.sh` (o `scripts\dev.ps1`).
- **Resultado esperado:** `bosquevivo setup` crea `backend/.env` y `web/.env.local`, genera `ADMIN_TOKEN`, `PHONE_HMAC_KEY`, `WHATSAPP_VERIFY_TOKEN` y `SESSION_SECRET`, elige datos abiertos si no hay Earth Engine, crea la base con 3 proyectos de demostración, conserva comentarios y valores existentes al repetirse, y lista solo las claves externas que faltan.
- **Automatización:** `integration/test_setup.py` (4 pruebas); prueba manual de `scripts/start.sh` en modo producción (API, sitio y chat en línea respondiendo).
- **Resultado:** Aprobado.

PostgreSQL, migración y despliegue

CP-59 Suite completa contra PostgreSQL

- **Requisito:** 3.3.6 Portabilidad, 3.3.3 Fiabilidad · **Prioridad:** 2
- **Pasos:** `TEST_DATABASE_URL=postgresql://… uv run pytest` sobre un PostgreSQL 16 vacío.
- **Resultado esperado:** las mismas pruebas que con SQLite pasan con PostgreSQL (tipos con zona horaria, largo de columnas, migración ligera, secuencias).
- **Resultado obtenido:** 330 de 330 aprobadas en PostgreSQL 16; la CI ejecuta las dos variantes en cada cambio.
- **Resultado:** Aprobado.

CP-60 Conexión a Supabase por el pooler

- **Prioridad:** 2
- **Resultado esperado:** `DATABASE_URL` con `[YOUR-PASSWORD]` se completa con `DATABASE_PASSWORD` codificada; `postgres://` y `postgresql://` usan el controlador psycopg; los hosts remotos exigen `sslmode=require`; en funciones sin estado se usa `NullPool` y sin sentencias preparadas (compatible con el *Transaction pooler*); la contraseña no aparece en los registros.
- **Automatización:** `unit/test_database_urls.py` (5 pruebas)
- **Resultado:** Aprobado. Verificado contra Supabase en producción (pooler `aws-0-us-east-1`).

CP-61 Migración de SQLite a PostgreSQL

- **Prioridad:** 2
- **Pasos:** `uv run bosquevivo migrate-db --from sqlite:///… --to postgresql://…`
- **Resultado esperado:** copia las 21 tablas en orden de dependencias, informa los valores que superan el largo de su columna antes de copiar, se niega a sobrescribir sin `--replace`, reinicia las secuencias y verifica los conteos; después se pueden crear registros nuevos.

- **Resultado obtenido:** 233 filas en 21 tablas copiadas a PostgreSQL 16; luego se crearon un usuario (id 3) y una auditoría (corrida 4) sin conflictos de identificador.
- **Automatización:** `integration/test_migrate.py` (2 pruebas)
- **Resultado:** Aprobado.

CP-62 API liviana y despliegue en Vercel

- **Prioridad:** 2
- **Pasos:** instalar solo el núcleo de dependencias, arrancar la API con `VERCEL=1` contra PostgreSQL y recorrer salud, proyectos, reportes, chat comunitario y solicitud de auditoría; luego `VERCEL_TOKEN=… scripts/deploy-vercel.sh`.
- **Resultado esperado:** el núcleo pesa menos de 500 MB; la API arranca sin las librerías de cálculo; las tareas se procesan en la misma petición; los reportes PDF y HTML se generan al vuelo; una auditoría pedida a la API desplegada falla con el mensaje de usar la CLI.
- **Resultado obtenido:** núcleo de 274 MB; API en modo Vercel respondió `/health`, proyectos, reporte PDF (200) y HTML (200), chat comunitario y la auditoría quedó «failed» con el mensaje esperado. `integration/test_inline_jobs.py` (4 pruebas).
- **Despliegue real (04/11/2026):** API en https://bosque-vivo-api.vercel.app y sitio en https://bosque-vivo-web.vercel.app sobre Supabase. Se detectó y corrigió una colisión al crear el esquema con varias instancias a la vez (INC-15). `push-db` subió 7.163 filas en 6 s; `/api/v1/projects` devolvió 58 proyectos; reporte PDF 200 en 1,1 s; chat en línea respondió el saludo y el aviso; `/proyectos` pasó de 15–25 s a 1–2 s con el endpoint `/api/v1/explorer` (INC-16).
- **Resultado:** Aprobado.

Informe de resultados de pruebas

Ejecución del plan de pruebas sobre la rama de trabajo que se fusionó a `main` el 03/11/2026. Entorno: Linux, Python 3.12, Node 22, Chromium con Playwright; base SQLite en memoria (pruebas automáticas) y bases temporales (E2E y mediciones).

1. Resumen

Indicador	Resultado
Casos especificados	62
Casos aprobados	58 (94 %)
Casos fallidos	0
Casos bloqueados por dependencias externas	4 (CP-29 y CP-46 del piloto, CP-49 WhatsApp y la evaluación formal de CP-50)
Requisitos funcionales RF01–RF09 con casos aprobados	9 de 9 (100 %)
Pruebas automáticas del backend	330 aprobadas, 0 fallidas, tanto en SQLite como en PostgreSQL 16
Cobertura de líneas del backend	88 % (5.438 sentencias)
Pruebas del sitio (Vitest)	94 aprobadas en 10 archivos
Pruebas del panel	5 aprobadas
Verificaciones E2E en navegador	30 de 30 aprobadas
Análisis estático	`ruff` y `pyright`: 0 errores; `eslint` y `tsc`: 0 errores
Compilación de producción del sitio	Correcta
Incidencias registradas	18 (todas cerradas; ver `incidencias.md`)

2. Resultados por suite

Suite	Comando	Resultado	Evidencia
Backend: estilo	`uv run ruff check . && uv run ruff format --check .`	Sin errores, 158 archivos con formato	`evidencias/backend-ruff.txt`
Backend: tipos	`uv run pyright`	0 errores	`evidencias/backend-pyright.txt`
Backend: pruebas y cobertura	`uv run pytest --cov=bosquevivo --junitxml=...`	330 aprobadas en 28 s, cobertura 88 %; las mismas 330 en PostgreSQL 16	`evidencias/backend-pytest.txt`, `backend-junit.xml`, `backend-postgres-junit.xml`
Sitio: estilo	`npm run lint`	Sin errores	`evidencias/web-lint.txt`

Suite	Comando	Resultado	Evidencia
Sitio: pruebas	`npx vitest run`	94 aprobadas	`evidencias/web-vitest.txt`, `web-junit.xml`
Sitio: compilación	`npm run build`	Correcta	`evidencias/web-build.txt`
Panel	`uv run pytest`	5 aprobadas	—
E2E	`docs/plan-de-pruebas/scripts/ejecutar-e2e.sh`	30 de 30	`evidencias/e2e-salida.txt`, `e2e-resultados.json`, `evidencias/capturas/`

Cobertura de los módulos nuevos o modificados

Módulo	Cobertura
`accounts/service.py` (RF09, 2FA, avisos, invitaciones)	98 %
`accounts/mailer.py`, `accounts/templates.py` (SMTP)	98 %
`setup.py` (configuración automática)	99 %
`api/routers/auth.py`	91 %
`accounts/passwords.py`	95 %
`core/incidents.py`	100 %
`satellite/ndvi.py`	100 %
`satellite/service.py`	100 %
`assistant/dialogue.py`	98 %
`assistant/service.py`	95 %
`assistant/community.py`	90 %
`registry/importer.py`	87 %
`api/routers/community.py`	100 %

La cobertura más baja corresponde a clientes de servicios externos que no se llaman en las pruebas por diseño (`satellite/providers/gee.py`, `assistant/client.py` de Meta y `ai/llm/gemini.py`).

3. Mediciones de requisitos no funcionales

Comando: `uv run bosquevivo medir-rnf --rounds <n> --projects 3`. El asistente se mide con las intenciones del ERS (estado, alertas, ayuda) y dos preguntas libres; sin clave de Gemini las preguntas libres usan la respuesta de respaldo.

Requisito	Meta	Datos simulados (`rnf-fake.json`)	Datos abiertos reales (`rnf-open.json`)	Cumple
3.3.1 Respuesta del asistente	≥ 90 % en < 10 s	100 consultas: p50 0,002 s, p90 0,005 s, máx. 0,009 s, 100 % < 10 s	20 consultas: p90 0,005 s, 100 % < 10 s	Sí (repetir con Gemini real)
3.3.1 Cálculo del índice	< 5 min por polígono	0,04–0,05 s	43,7 s, 44,7 s y 62,9 s (MODIS 250 m + Hansen, descarga real)	Sí
3.3.3 Determinismo	Mismo índice con los mismos insumos	99,9/99,9; 100/100; 100/100	97,2/97,2; 96,0/96,0; 56,1/56,1	Sí
3.3.3 Incidentes críticos	≤ 2 al mes	—	—	Se mide en el piloto (CP-46)
3.3.4 Disponibilidad	95 % en el horario del piloto	—	—	Se mide en el piloto (CP-46)

4. Pruebas de extremo a extremo

`scripts/ejecutar-e2e.sh` crea una base temporal con los proyectos de demostración, invita una cuenta de auditor (`bosquevivo user-invite`), activa la verificación en dos pasos con el buzón local (`AUTH_TWO_FACTOR=on`), levanta la API y el sitio compilado, y recorre en Chromium:

Flujo	Casos	Resultado
Invitación: correo con enlace, bienvenida, primera contraseña	CP-56	3 verificaciones aprobadas
Recuperación de contraseña (RF09): acceso restringido, credenciales inválidas, mensaje genérico, enlace por correo, token inválido, contraseñas distintas y débiles, actualización, enlace reutilizado	CP-30 a CP-36	10 verificaciones aprobadas
Verificación en dos pasos y avisos: código por correo, código incorrecto, reenvío, entrada con la contraseña nueva, sesión con el correo, aviso de inicio de sesión, aviso de cambio de contraseña, correos con HTML	CP-37, CP-53 a CP-55	8 verificaciones aprobadas
Chat en línea (RF07, RF08, Ley 1581): saludo y aviso, consentimiento, ayuda, registro del proyecto, estado, lenguaje sencillo, alertas, consulta no reconocida, borrado	CP-21 a CP-28, CP-42	9 verificaciones aprobadas

Capturas: `evidencias/capturas/00-invitacion-activada.png` a `08-codigo-2fa.png`.

Prueba manual adicional (CP-58): `scripts/start.sh` en un clon sin `.env` generó los secretos, la base y los datos de demostración, compiló el sitio y dejó la API (`/health`), el sitio y el chat en línea respondiendo en modo producción.

4.1 PostgreSQL y despliegue (versión 1.2)

Prueba	Resultado
Suite completa contra PostgreSQL 16 (`TEST_DATABASE_URL`)	330 de 330 aprobadas
`migrate-db` de una base SQLite con datos de demostración, cuentas e invitaciones	233 filas en 21 tablas; registros nuevos sin conflicto de secuencias
Entorno solo con el núcleo de dependencias	274 MB (límite de Vercel: 500 MB)
API en modo Vercel (`VERCEL=1`) contra PostgreSQL	`/health`, proyectos, reporte PDF y HTML generados al vuelo, chat comunitario; auditoría rechazada con el mensaje de usar la CLI
Producción (Vercel + Supabase)	Sitio https://bosque-vivo-web.vercel.app y API https://bosque-vivo-api.vercel.app; 58 proyectos cargados con `push-db`; `/proyectos` en 1–2 s; reporte PDF, chat en línea y acceso responden

5. Evaluación de los criterios de aceptación

Criterio (plan, 4.5.1)	Resultado	Cumple
1. 100 % de pruebas automáticas aprobadas y análisis estático sin errores	330 (SQLite y PostgreSQL) + 94 + 5 aprobadas, 0 errores	Sí
2. Cobertura del backend ≥ 80 %	88 %	Sí
3. RF01–RF09 con al menos un caso aprobado	9 de 9	Sí
4. Casos de prioridad 1 y 2 ejecutados	Todos, salvo CP-29 (usuarios del piloto)	Sí, con la excepción prevista
5. 0 incidencias críticas o altas abiertas	0 abiertas	Sí
6. Rendimiento	100 % < 10 s; máximo 62,9 s por polígono	Sí
7. Determinismo	3 de 3	Sí
8. Compilación de producción	Correcta	Sí

Conclusión. Se cumplen los criterios para dar por completado el plan en esta etapa. Quedan cinco casos o partes bloqueados por dependencias externas (credenciales de Meta, de Google AI Studio y de correo SMTP, y usuarios del piloto); tienen guía de ejecución en `docs/guias` y en este plan, y no afectan los requisitos funcionales comprometidos, que se verificaron sin ellos.

6. Cómo repetir la ejecución

```
cd backend
uv run ruff check . && uv run ruff format --check . && uv run pyright
uv run pytest --cov=bosquevivo
SATELLITE_PROVIDER=open uv run bosquevivo medir-rnf --rounds 100 --projects 3
```

```
cd web
npm run lint && npx next typegen && npm run typecheck && npm test && npm run build
```

```
bash docs/plan-de-pruebas/scripts/ejecutar-e2e.sh
```

Registro de incidencias

Incidencias encontradas al contrastar el sistema con el ERS (20/10/2026) y al ejecutar el plan de pruebas el 03/11/2026. Severidad: crítica, alta, media, baja. Todas quedaron cerradas y verificadas con la suite completa (18 incidencias).

ID	Caso	Requisito	Descripción	Severidad	Causa	Corrección	Estado
INC-01	CP-02	RF01	Un KML con atributos en `Schema`/`SchemaData` se leía sin `project_id` ni fecha, y la fila se rechazaba por falta de identificador	Media	El controlador LIBKML de GDAL expone los atributos de `<Data>` pero no siempre los de `SchemaData` sin esquema resoluble	Se documenta el formato admitido (`ExtendedData`/`Data`) y se añaden alias de fecha (`begin`, `timestamp`, `registration_date`, `fecha_registro`)	Cerrada
INC-02	CP-30	RF09	`/acceso` usaba `Date.now()` durante el prerender y Next mostraba un error en desarrollo	Baja	Con Cache Components la verificación de la sesión debe ejecutarse en tiempo de solicitud	`await connection()` antes de leer la sesión	Cerrada
INC-03	CP-21	RF07	En el chat en línea las burbujas de la persona ocupaban todo el ancho	Baja	Faltaba `w-fit` en el párrafo	Estilo corregido	Cerrada
INC-04	CP-25	RF07	ALERTAS decía «Todavía no tenemos datos de pérdida» para un proyecto con línea base pero sin pérdida registrada	Media	Se confundía «sin filas de pérdida» con «sin datos»	Si hay línea base y no hay pérdida: «Sin alertas: no se detectó pérdida de bosque»	Cerrada

ID	Caso	Requisito	Descripción	Severidad	Causa	Corrección	Estado
INC-05	CP-04, CP-05	RF01	La importación genérica aceptaba filas sin fecha de registro, reparaba geometrías inválidas en silencio y sobrescribía identificadores existentes	Alta	El importador solo tenía el modo permisivo pensado para la base de Karnik	Modo estricto por defecto en CLI y API, con motivo de rechazo por fila; `--lenient` conserva el comportamiento anterior	Cerrada
INC-06	CP-11	RF03	Los valores de NDVI fuera de [−1, 1] se recortaban en lugar de descartarse	Alta	`np.clip` en el cálculo	Se descartan (quedan sin dato) y los puntos inválidos se registran como escena inválida	Cerrada
INC-07	CP-26	RF07	Sin `GEMINI_API_KEY` el chat comunitario y WhatsApp fallaban por completo (503)	Alta	El proveedor de IA se construía de forma obligatoria	`UnavailableLLM`: las intenciones del ERS responden sin IA y las preguntas libres usan el respaldo y la ayuda	Cerrada
INC-08	CP-08	RF02	El umbral de nubosidad por escena no era configurable (solo existía la máscara por píxel)	Media	Requisito no implementado	`CLOUD_MAX_PERCENT` (20 % por defecto) filtra Sentinel-2 y Landsat y queda en la trazabilidad	Cerrada
INC-09	CP-09	RF02	Cuando no había escenas no quedaba registro de la incidencia	Media	Requisito no implementado	Registro de incidentes con origen y severidad (`core/incidents.py`)	Cerrada

ID	Caso	Requisito	Descripción	Severidad	Causa	Corrección	Estado
INC-10	E2E	Entorno de pruebas	Entre corridas del E2E quedaba vivo el servidor hijo de `next start` y la siguiente corrida usaba el servidor viejo	Baja	`kill` del proceso `npx` no termina su grupo	El script usa `setsid`, termina el grupo de procesos y verifica que los puertos estén libres	Cerrada
INC-11	E2E	Entorno de pruebas	El script E2E contaba el texto de bienvenida del chat como una burbuja de respuesta	Baja	Selector demasiado amplio	Atributo `data-from` en cada burbuja	Cerrada
INC-12	CP-56	RF09	Al activar una cuenta invitada, el enlace no quedaba marcado como usado y podía reutilizarse	Alta	La lista de enlaces del usuario estaba en la caché de la sesión de SQLAlchemy y no incluía el recién creado	Los enlaces abiertos se consultan en la base y el enlace usado se marca de forma explícita	Cerrada
INC-13	CP-53	RF09	Con la verificación en dos pasos siempre activa, sin SMTP nadie podía iniciar sesión (el código quedaba en el buzón local)	Alta	El segundo factor no consideraba si el correo se puede entregar	`AUTH_TWO_FACTOR=auto`: se activa solo con SMTP; `on`/`off` lo fuerzan	Cerrada
INC-15	CP-62	3.3.3	Al primer despliegue, varias instancias de Vercel crearon las tablas a la vez y Supabase respondió `UniqueViolation` en el catálogo; la API devolvía 500	Alta	`CREATE TABLE` concurrente sin coordinación	Candado `pg_advisory_xact_lock` alrededor de la creación del esquema; probado con 8 arranques simultáneos	Cerrada

ID	Caso	Requisito	Descripción	Severidad	Causa	Corrección	Estado
INC-16	CP-62	3.3.1	En producción `/proyectos` tardaba 15–25 s	Media	El sitio hacía 116 llamadas a la API (polígono y auditoría por proyecto), cada una con su conexión a la base	Endpoint `/api/v1/explorer` que entrega todo en una llamada; ahora 1–2 s	Cerrada
INC-17	CP-50	3.3.1	Con Gemini real, el chat tardó 78–111 s y a veces cayó al respaldo	Alta	`gemini-flash-latest` respondía 503/504 por alta demanda; cada paso del agente reintentaba el mismo modelo hasta 4 veces	Cambio inmediato a otro modelo gratuito (`gemini-flash-lite-latest`), pausa de 3 minutos al modelo saturado y tiempo límite de 20 s por llamada; ahora 3–5 s por pregunta	Cerrada
INC-18	CP-51	RA07	La herramienta `buscar_conocimiento` nunca encontraba nada (metodología, glosario, normativa)	Alta	El corpus no se indexaba en la tabla `kb_chunk` en ningún flujo	Se indexa automáticamente en la primera búsqueda	Cerrada
INC-14	CP-56	Entorno de pruebas	El E2E leía la bienvenida de la invitación antes de que la página terminara de cargar	Baja	Contenido transmitido con Suspense	Se espera el formulario antes de verificar	Cerrada

Observaciones que no son defectos

- En desarrollo (`next dev`), tras recargar en caliente el encabezado, React informó un desajuste de hidratación. No se reproduce con la compilación de producción ni en una carga nueva; es propio de la recarga en caliente.
- Las respuestas libres del asistente sin clave de IA usan el texto de respaldo («No puedo responder con seguridad…»). Es el comportamiento esperado (ADR 0009) y queda cubierto por CP-26.